Rugby Intelligent Camera

1st Canto da Silva Delatorre, Tiago
Electrical and informatics department
INSA Toulouse
Toulouse, France
tcanto-da-si@insa-toulouse.fr

2nd Tassin, Nicolas
Electrical and informatics department
INSA Toulouse
Toulouse, France
tassi@insa-toulouse.fr

3rd Visbal Pinzon, Kevin Felipe
Electrical and informatics department
INSA Toulouse
Toulouse, France
kvisbal-pinz@insa-toulouse.fr

4th Oliveira Donzelli, Gabriela
Electrical and informatics department
INSA Toulouse
Toulouse, France
goliveira-do@insa-toulouse.fr

5th Herrati, Noam
Electrical and informatics department
INSA Toulouse
Toulouse, France
herrati@insa-toulouse.fr

6th Goyheneix, Florian
Electrical and informatics department
INSA Toulouse
Toulouse, France
goyheneix@insa-toulouse.fr

7th Guarni, Wissal
Electrical and informatics department
INSA Toulouse
Toulouse, France
guarni@insa-toulouse.fr

Abstract

Amateur sports clubs lack access to professional multi-camera video analysis systems due to the high cost and technical complexity of commercially availabe solutions. This gap limits the ability of coaches and analysts to review match footage from multiple angles in a systematic and automated way. This work presents a low-cost automated camera switching system designed for rugby match coverage, built around a two-smartphone acquisition setup and a computer vision pipeline running on consumer-grade hardware. The proposed system uses YOLOv8n for player and ball detection, combined with a density-based scoring algorithm to evaluate and select the most relevant camera feed for each frame. Real-time video transmission was investigated but could not be validated due to hardware constraints. Validation was conducted on static images extracted from real match footage, confirming that the detection and scoring logic produces coherent camera selection results. The results demonstrate the feasibility of the approach within a minimal-budget context, and indicate that access to dedicated hardware and a domain-specific detection model would be sufficient to bring the system to real-time operational capability.

***Index Terms*—computer vision, rugby, cost-effective, camera tracking, YOLO**

Introduction

The growing availability of computer vision and deep learning technologies has transformed sports broadcasting and performance analysis. Camera-based tracking systems are now widely used in professional contexts, combining multi-camera setups with real-time image processing to follow players and the ball throughout a match. However, such systems remain complex and expensive, making them largely inaccessible to amateur clubs operating with limited resources. Several approaches have been proposed to automate sports tracking and broadcasting. YOLO-based architectures have demonstrated strong performance for real-time player and ball detection in various sports contexts(), and automated camera selection methods have been developed using spatial and temporal cues to identify the most relevant viewpoint at each moment (). Despite these advances, existing solutions have been designed primarily for professional environments and have not been adapted to the specific constraints of amateur sport. Two gaps remain unaddressed. First, no affordable, fully integrated tracking and broadcasting system has been proposed for amateur clubs. Second, ball detection in rugby presents a particularly underexplored challenge: frequent occlusions by players and an irregular ball trajectory make reliable real-time detection significantly harder than in sports such as football or basketball. To date, no study has investigated these two problems jointly. This project aimed to develop a cost-effective, autonomous system capable of capturing, processing, and broadcasting amateur rugby matches in real time, combining multi-camera recording, YOLO-based player and ball detection, and a dynamic camera selection module.

I. Related Work

Sports analysis has evolved through AI and computer vision, yet high costs exclude amateur clubs. This work establishes the theoretical foundation for a project aimed at democratizing access to automated video tracking in amateur rugby.

A. Object Detection and Capture Systems

To automate capture, the market uses panoramic stitching **taxbol˙computer-implemented˙2022** or robotic mounts **linderoth˙system˙2020**. Real-time detection relies on YOLO-NAS for optimized latency **biro˙predictive˙2023**. Because fast-moving balls cause blur, systems integrate temporal context from adjacent frames rather than relying on single images **xu˙yolo-based˙2025**, **biro˙predictive˙2023**.

B. Trajectory Prediction and Tracking Logic

To track these detections, the Kalman Filter method predicts trajectories, maintaining continuity during occlusions. Fuzzy logic optimizations often refine these predictions to handle abrupt athletic changes, ensuring algorithms correctly link new data to existing tracks without identity switches **yousefi˙tracking˙2023**.

Robust visual tracking relies on the integration of detection, temporal prediction, and dynamic state estimation rather than on perfect frame-by-frame detection. Al-Hadrusi and Sarhan **AlHadrusi2012PTZ** propose a decision-based framework combining tracking, prediction, and quality evaluation (confidence, visibility, occlusions, resolution), enabling tracking continuity during short-term observation losses. These principles are directly applicable to rugby ball tracking with a fixed camera, where the ball is small, fast-moving, and frequently occluded.

C. Spatiotemporal Feature Extraction

To enable real-time camera following, transfer learning is used within 3D Convolutional Neural Network (3D-CNN) architectures. Inflated 3D Networks (I3D) **xie˙rethinking˙2018** is used to process spatiotemporal information. By inflating pretrained 2D filters into the temporal domain, I3D models overcome the computational bottlenecks of traditional 3D-CNNs, optimizing the balance between recognition accuracy and response latency.

Semantic Segmentation assigns pixel-level classes to isolate game elements utilizing deep models like VGG-16 with dilated convolutions**li˙research˙2024**, **xie˙rethinking˙2018**. This approach handles class imbalance via modified cross-entropy loss, achieving high Mean IOU scores. Optical Flow quantifies motion dynamics, evolving from sparse Lucas–Kanade and dense Horn–Schunck methods**opencv˙optical˙flow˙tutorial**, **scaler˙motion˙estimation˙optical˙flow˙2025**, **viso˙optical˙flow˙2024** to deep architectures (FlowNet, PWC-Net) that maintain accuracy under occlusions. Action and Event Recognition combines CNN-based spatial features with LSTM-based temporal modeling to detect events in near real-time **poms˙scanner˙2018**, **xu˙yolo-based˙2025**, with robustness enhanced by multi-modal inputs like ball trajectories **wu˙multi-camera˙2020**, **zhang˙multi-camera˙2020**.

D. Camera Localization and State Estimation

Finally, Camera Localization employs a two-stage PTZ calibration approach **AlHadrusi2012PTZ**. By utilizing deep descriptors (SuperPoint, SuperGlue) and the PTZ-IBA algorithm to minimize reprojection errors**Labbe2010NonlinearObserver**, **Xavier2001VisualServoing**, the system achieves high precision with efficient online processing**Pix4Team**.

In monocular settings, Xavier *et al.* **Xavier2001VisualServoing** show that effective tracking can be formulated directly in the image plane using dynamic estimation corrected by visual feedback, without explicit 3D reconstruction. Complementarily, Labbé *et al.* **Labbe2010NonlinearObserver** ground tracking in nonlinear observer theory, combining noisy visual measurements with dynamic models to maintain coherent state estimates (position and velocity) during occlusions or degraded observations.

E. Environmental impact

The environmental impact of artificial intelligence is a growing concern, as the energy consumption and carbon footprint of large-scale data centers have become significant bottlenecks for sustainable cloud computing. *et al.* **Labbe2010NonlinearObserver** To address these ecological and performance challenges, recent studies have focused on accelerating inference directly on edge devices using optimization tools like NVIDIA TensorRT *et al.* **Labbe2010NonlinearObserver** , which streamlines model execution to achieve high frame rates on local hardware.

F. Design Synthesis for Embedded Systems

To enable affordable rugby analysis on a Raspberry Pi, this project prioritizes computational efficiency. We conclude that a faster tracking method like YOLO-NAS is the optimal detection model due to its speed, while Kalman Filtering provides essential trajectory prediction during fast motion or occlusions. We reject complex 3D reconstruction in favor of robust 2D tracking directly on the image plane. This combination of lightweight detection and mathematical prediction ensures the system maintains real-time performance despite the hardware's strict processing limits.

II. Experiments and Results

A. Camera Positioning

1) Experimental Setup: The strategic positioning of cameras is a critical factor as it directly impacts both hardware requirements and software performance. Increasing the number of cameras allows for closer views of the action, which significantly enhances the detection of key points of interest. However, this architectural choice creates a trade-off: a higher camera count increases the volume of video streams to be processed, thereby raising the complexity of the hardware infrastructure. Furthermore, the choice of the detection model, such as YOLO (You Only Look Once), plays a pivotal role; since YOLO is available in multiple versions with varying computational costs, a configuration with fewer cameras may allow for the deployment of lighter, less resource-intensive models. To find the optimal balance between detection accuracy and computational efficiency, we conducted field tests on a rugby pitch, evaluating various configurations.

To determine the most efficient layout for player tracking, we evaluated three distinct spatial configurations. The first configuration consisted of a linear array where all cameras were positioned parallel to the pitch's width, mounted exclusively on one side of the field to provide a consistent viewing angle.

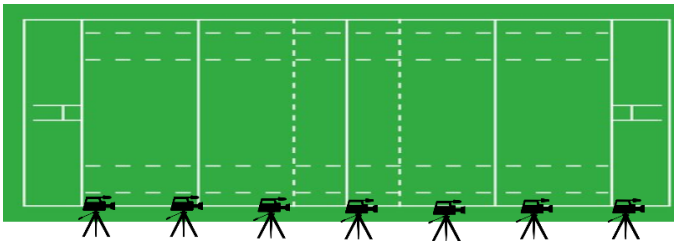


Fig. 1. First camera setup

The second setup involved placing cameras at the center of each half-pitch; these units were oriented back-to-back at 45-degree angles to maximize the combined field of view from a central vantage point.

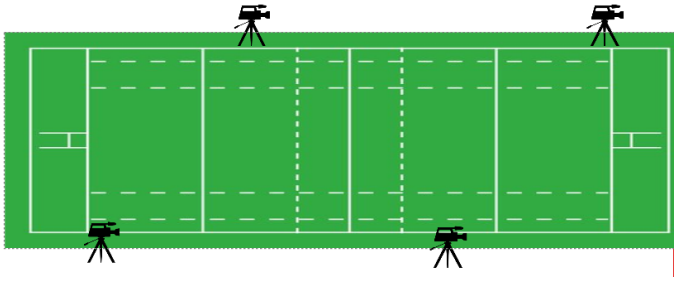


Fig. 2. Second camera setup

Finally, we tested a staggered configuration, alternating the camera positions on both sides of the pitch along its length. This alternating approach was designed to minimize occlusions and assess the impact of multi-perspective depth acquisition on the detection accuracy of the YOLO model.

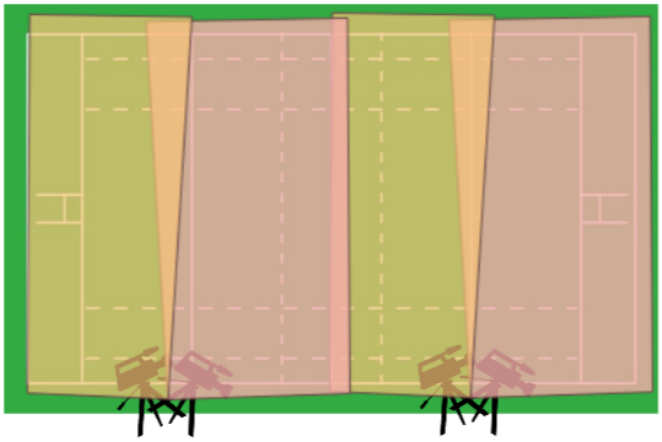


Fig. 3. Third camera setup

2) Results and Analysis: The evaluation of the three configurations revealed significant disparities in terms of resource efficiency and practical feasibility. The first setup, requiring seven cameras to achieve full pitch coverage, provided the highest spatial precision. However, this high density of devices leads to a prohibitive computational load, necessitating a heavy YOLO model architecture to process multiple simultaneous streams. Furthermore, from a broadcast perspective, the frequent camera switching required to follow the action results in a fragmented and suboptimal viewing experience for streaming audiences.

The second configuration proved to be the most balanced alternative, requiring only four cameras. While it offers slightly lower precision than the first setup, it successfully covers the entire playing area at a significantly lower hardware cost. Crucially, this layout allows for smoother tracking of player movements, providing a more fluid and continuous visual experience for live broadcasting.

Finally, the third configuration, also utilizing four cameras, presented a compelling advantage by keeping the ball in the foreground for a majority of the match. Nevertheless, practical constraints make this setup difficult to implement in real-world stadium environments. The presence of spectators along the touchline and the frequent movement of substitutes on the sidelines introduce significant occlusion risks, which would frequently obstruct the cameras' line of sight and degrade the reliability of the detection model.

To determine the most efficient layout for player tracking, we evaluated three distinct spatial configurations. The first configuration consisted of a linear array where all cameras were positioned parallel to the pitch's width, mounted exclusively on one side of the field to provide a consistent viewing angle. The second setup involved placing cameras at the center of each half-pitch; these units were oriented back-to-back at 45-degree angles to maximize the combined

field of view from a central vantage point. Finally, we tested a staggered configuration, alternating the camera positions on both sides of the pitch along its length. This alternating approach was designed to minimize occlusions and assess the impact of multi-perspective depth acquisition on the detection accuracy of the YOLO model.

3) Discussion:

Considering the available computational resources and the specific context of non-professional matches where spectators often stand in close proximity to the touchline.Configuration 2 was ultimately selected as the most viable solution. This setup offers an optimal compromise, maintaining a low camera count while ensuring a comprehensive view of the play. Nevertheless, further optimizations are possible; specifically, increasing the mounting height of the cameras would significantly enhance the system's performance. A more elevated perspective would reduce the perspective distortion for distant actions and further mitigate the risk of intermittent occlusions caused by individuals on the sidelines.

B. Video Acquisition and Transmission Architecture

Real-time automated tracking of amateur rugby players imposes strict constraints regarding bandwidth, latency, and hardware costs. This section details the technological choices made to ensure a fluid and synchronized video stream to the central processing unit.

1) Capture Device Choice:

Several capture solutions were evaluated to meet the requirements of the multi-camera tracking project. The use of cameras on motorized platforms was initially considered for its dynamic tracking capability which could have reduced the number of cameras needed and thus the total cost. However, extreme sensitivity to control delays and prohibitive costs led to its rejection. Similarly, solutions based on microcontrollers such as Arduino, while inexpensive, offered insufficient acquisition performance and resolution for exploitation via YOLOv8 detection models.

Ultimately, smartphones were selected as the capture units. These devices offer an optimal compromise:

- **Image Quality :** High-resolution sensors capable of providing images that can be processed by the YOLO model
- **Cost :** Utilization of existing hardware, drastically reducing the initial investment.
- **Wireless Capability :** 4G/5G cellular and Wi-Fi connection are embedded.

2) Evaluation of Streaming Protocols:

To route the video streams from the smartphones to the processing computer, three main protocols were compared:

Protocol	**Infrastructure**	**Latency**	**Stability / Cost**
SRT	4G / 5G	200 ms – 500 ms	Excellent; high subscription costs
RTMP	4G/5G + remote server	2000ms	Industry standard (Youtube, Twitch); requires a server
HTTP (Local)	Wi-Fi (router)	**50 ms – 150 ms**	Excellent stability; no recurring cost

The **HTTP protocol over a local network** was selected. Unlike cellular solutions (SRT/RTMP) that depend on public network load and incur data costs, the local network allows full control, high stability, and minimizes latency.

3) Local Network Implementation and Topology:

The network architecture put in place relies on the creation of a **high-performance private Wi-Fi network** (using a HUAWEI WiFi AX3-type router). This configuration makes it possible to avoid potential interference with the public networks of stadiums.

The system is structured as follows:

1) **Wireless transmission:** Two smartphones transmit their respective streams to the central access point via the HTTP protocol. Tests in real conditions validated an effective range of **80 metres**, covering a significant portion of the playing surface.
2) **Downstream link:** The router is connected to the processing computer (YOLOv8 computing unit) via a gigabit Ethernet connection. This wired link ensures that the bottleneck does not occur at the final data reception stage.
3) **Centralised processing:** The computer simultaneously receives the four streams to perform player detection and tracking.

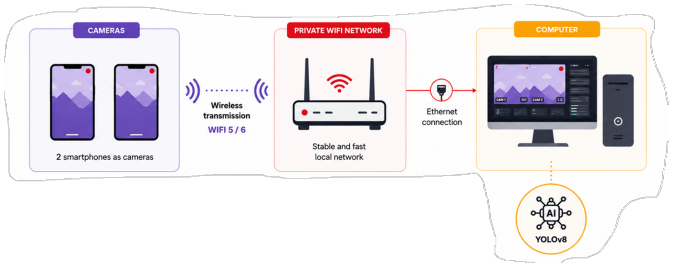


Fig. 4. Network topology: two smartphones transmit wirelessly via HTTP to a private Wi-Fi router, which forwards data via gigabit Ethernet to the processing computer.

This communication layer serves as the foundation for the Camera Score Algorithm, which must ingest these concurrent streams and dynamically prioritize the optimal feed based on the proximity of the tracked player or ball to the camera's optical axis.

C. YOLO Model

A model of Artificial Inteligence is a trained code to learn paterns based on a database[citar qualquer coisa que defina um modelo]. In between the existent models, there is the model YOLO whose algorithyme treats the image only one time (You Only Look Once).

Models like YOLO have faster responses to the video's processing, so they are usually used for real time processing, autonomus cars, monitoring, etc. The detection of plany of objects at the same time is also a caracteristc of these models, but the main reason we chose YOLO as our model was to process efficiently and with high speed.

To be able to use YOLO in python, we used the library Ultralytics which makes simpler the utilisation of pre-trained models, image detection and others uses of computer vision. As so, the Figure5**ultralytics˙models** compares the diferentes versions of YOLO's models by latency and precision of detection. Therefore, considering the speed as main goal to track in real time during a rugby match, we chose to work with YOLOv8 version.

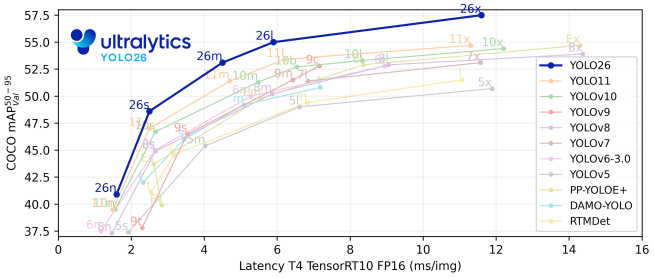


Fig. 5. Comparaison des modèles YOLO supportés par Ultralytics. Source : Ultralytics**ultralytics˙models**

Nevertheless, the Ultralytics website also presents the performance metrics of differents models of the chosen version as shown at the Figure 6**ultralytics˙models**. To the image processing, tests were made to choose the one with good balance between accuracy and processing time.

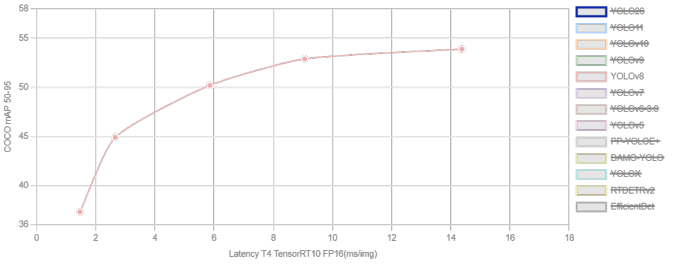


Fig. 6. Courbe de performance des différentes variantes du modèle YOLOv8. Source : Ultralytics**ultralytics˙models**

The version of the chosen model was decided by made using one frame taken from a video recorded in the real situation of a amateur match, the goal of this article, in each of the models offered by Ultralytics. The Table I, shows the results obtained in terms of processing time and quality of detection.

TABLE I
COMPARISON OF YOLOV8 MODEL VARIANTS IN THE SAME FRAME.

Model	Detected Objects	Inference	Total Time	FPS
YOLOv8n	15 persons	61.8 ms	66.0 ms	15.15
YOLOv8s	18 persons	137.5 ms	140.8 ms	7.10
YOLOv8m	21 persons	206.2 ms	208.8 ms	4.79
YOLOv8l	22 persons	616.8 ms	621.5 ms	1.61
YOLOv8x	22 persons	653.5 ms	657.2 ms	1.52

Based on these results and aiming for fast processing, we chose YOLOv8n because, even though it does not recognize as many players as the more robust models, it is more than ten times faster.

Another important conclusion is that increasing the size of the YOLOv8 model does not necessarily lead to a proportional improvement in detection performance like YOLOv8x which does not perform more efficiently on the image than YOLOv8l, even though it requires more processing time.

Although the larger models detect more players, their processing time increases significantly, which makes them less suitable for real-time video analysis.

1) Methods: To track the focus of a rugby match, we agreed that the most important thing we should track is the ball, but in rugby the ball is small and always hidden between the players or very close to their chest, causing a big problem for the model's identification of the ball. In order to solve this problem, we added the class player; this implies that the players will also be identified by the model.

In rugby, the teams usually place themselves in lines of attack and defense, so, normally the point of interest will take place where the greatest number of players are. With both classes, we can already create a working tracking model.

The world of computer vision developers is growing and one of the biggest communities is the Roboflow Universe, which we can find a great number of models and datasets. The project Rugby Detection Computer Vision Model **rugby˙ruck˙detection** was the closest to our project's needs, and the pictures used in this dataset are similar to the images we captured in an INSA AS match.

2) Results: After testing the model from the project **rugby˙ruck˙detection** in some images from the captured match (Fig. 7 to Fig. 9), positive results were achieved: the model responds well to the images, identyfing a big number of player and the ball. Some problems were: confusing the arbitrator with a player, not identifying all the players, and sometimes not seeing the ball.

Unfortunately, the model **rugby˙ruck˙detection** isn't accessible to download, so it wouldn't be possible to use in a real time application, because for every frame the Roboflow repository would have to be accessed on the internet, increasing the cost and delay of the image processing.

To solve the presented problem, the project's dataset **rugby˙ruck˙detection** was cloned and a new AI model was trained, hoping to achieve the same results as the previous. Four models were created, two with yolov8n, one with yolov8s

Fig. 7. Test image 1, used model from Roboflow**rugby˙ruck˙detection**.

Fig. 10. Test image 1, used trained yolov8n model.

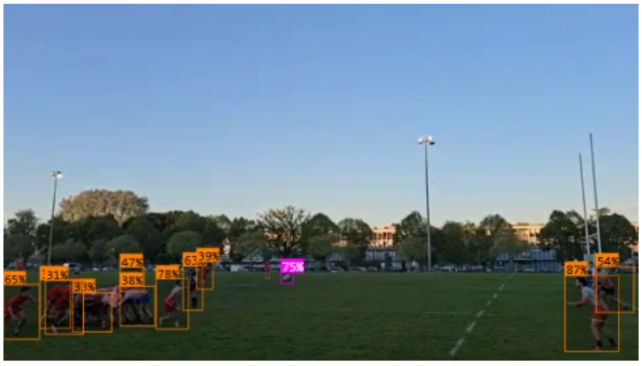


Fig. 8. Test image 2, used model from Roboflow**rugby˙ruck˙detection**.

Fig. 11. Test image 2, used trained yolov8n model.

and the last with yolov8m; they were all tested in an image not from the dataset.

These models aren't as good as the previous one because they don't have the same accuracy and have a greater difficulty of identifying the wanted features, as seen in Fig. 10 to Fig. 12. At least they are functional for video analyses and can be used for real-time applications.

To validate our models, we tested it in a video of an INSA AS match, but the power of the available GPU wasn't enough to analyze a nineteen second video (with 1141 frames). So, we couldn't validate our models in a hole video but, analyzing in a smaller video, we can see some problems of players and ball identification, showing how the project would look like in a real-time solution.

3) Conclusion: Regarding the YOLO model we can conclude that this solution works for identification of our needs to choose the best camera but, trying to find a low-cost solution for the amateur clubs we arrived at a lack of computing power that probably will increase the cost of this

Fig. 9. Test image 3, used model from Roboflow**rugby˙ruck˙detection**.

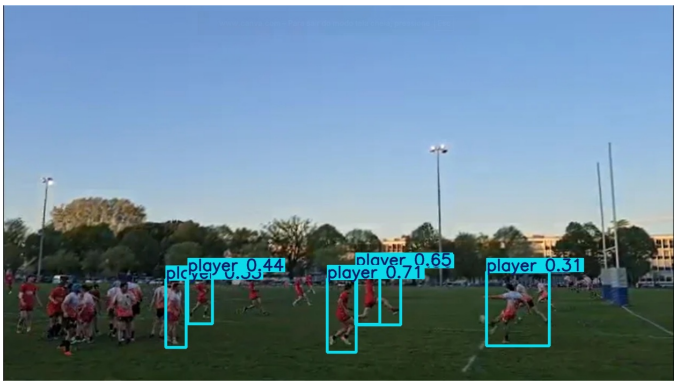


Fig. 12. Test image 3, used trained yolov8n model.

approach.

4) Improvements: To improve this project, we plan on using another YOLO model**roboflow˙tutorial˙2024** to see the statistics of the game as well, identifying tackle, penalty kick, throw-in, scrum, and other similar rugby situations that our other model can't identify. Another aimed improvement is to create a system**roboflow˙tutorial˙2024** so the model could be able to after identifying the players, separate them between team A and team B. The system consists of a Sigmoid loss language image pre-trained**roboflow˙tutorial˙2024** (extract color information from the player's shirt), associated with an UMAP**roboflow˙tutorial˙2024** (compact the previous information) and a Kmeans **roboflow˙tutorial˙2024** (sort both teams). These improvements would highly increase the quality of the performance analysis of the teams.

D. Phase test - Optimization of the performance

During the test phase of the Yolov8 based solution, the performance issue occurs. Indeed, running AI software on a computer requires strong configuration. On one hand, the processor of the machine is highly involved during the computation, but on the other hand, the graphics processor is even more used. It is explained by the fact that YOLO is based on matrix calculation **xu˙yolo-based˙2025** which is the main function of a graphical card.

We carried out our tests with a private individual's configuration composed of a Ryzen 3 3100 and an RTX 4060. The primary limitation of our setup is that the computer configuration is unbalanced; because of the processor, which is way older than the graphic card, with a better CPU we can easily expect around 40% performance gain.

To develop a real-time solution, we aim to maximize the frame rate (abbreviated as fps). Using a 30 fps camera, the number of cameras usable is $x/30$, with x being the average fps score when running YOLO on the computer.

First, the chosen YOLO model affects heavily the performance as described on this chart:

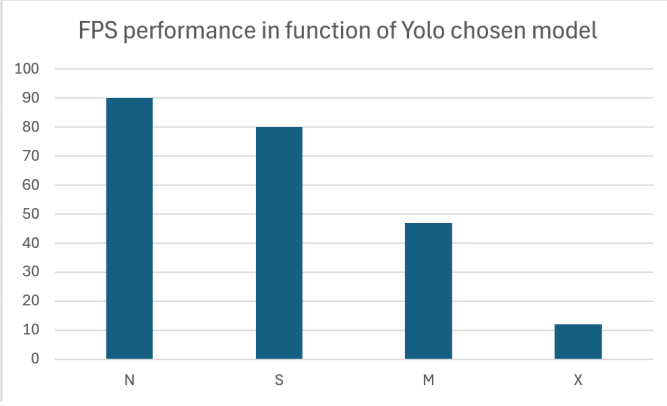


Fig. 13. FPS performance in function of YOLO chosen model

Those results seem to be sufficient, however due to blurs and bugs in the YOLO execution, it can be interesting to allocate a higher security margin about performance.

So, more fps is needed. A possibility of improving the amount of fps is to use an Nvidia tool: NVIDIA TensorRT. It is a software tool created by NVIDIA that optimizes AI models so they can run much faster and more efficiently on their graphics cards **tensorrt˙edge˙2024**. We used it to translate the file containing the coefficients used by YOLO to compute the result: the `.pytorch` file into an `.engine` file which is the same object but optimized to work better on Nvidia graphic cards.

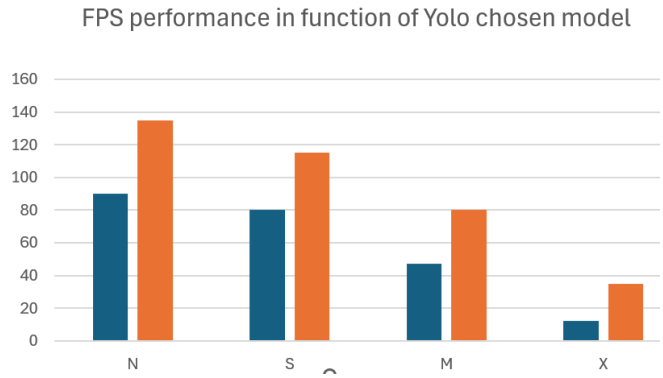


Fig. 14. FPS performance comparison: `.pytorch` (blue) vs `.engine` (orange)

Here in this chart (in orange) the performance of using this method compared to the original performance when using a `.pytorch` model (in blue).

Although deploying our YOLO-based solution on a remote server could have bypassed our local hardware constraints, we deliberately opted against a cloud-centric approach due to three major concerns. First, continuously streaming video data to remote servers introduces significant ecological issues, primarily driven by the high energy consumption and carbon footprint of large-scale data centers. Second, processing video streams off-site raises critical ethical and data privacy challenges, as sensitive information must be entrusted to third-party infrastructures. Finally, adopting this architecture creates a strong vendor lock-in, rendering the solution entirely dependent on large technology corporations and vulnerable to their proprietary systems, pricing models, and potential service outages.

Even though the RTX pro 6000 in a remote server cloud offers amazing performance around 611% of frame per second and for 422% lower power consumption compared to our local solution, we won't use it.

E. Camera score algorithm

1) Experimental Setup: Since the system architecture relies on multiple synchronized video streams, an intelligent selection algorithm is required to determine which camera feed should be broadcasted at any given moment. To achieve

this, we developed a dynamic scoring heuristic that assigns a confidence value between 0 and 100 to each camera. This score is calculated based on the output of the YOLO detection model, specifically targeting high-interest events such as the ball's position or dense clusters of players (scrums or rucks). By prioritizing the camera with the highest score, the system ensures that the streaming output remains focused on the core action of the match. Due to current hardware limitations and to optimize the algorithm's parameters, our initial development phase was conducted using static image datasets rather than live video streams. This preliminary approach allowed us to calibrate the scoring system and refine the detection thresholds without the computational overhead of real-time processing. By analyzing high-resolution snapshots from various field positions, we were able to identify the key visual features required for an accurate camera selection such as bounding box confidence and spatial distribution before transitioning to the more demanding requirements of continuous video analysis.

In the initial image-based testing phase, our primary objective was to detect the ball and assign a higher priority score to cameras where it appeared closest to the center of the frame. However, empirical observations showed that the ball is frequently occluded by players or difficult to detect due to its small size and high velocity. To address this, we implemented a secondary criterion based on the proximity of players to the camera. By calculating the average distance and spatial density of the detected bounding boxes, the system can infer the center of the action even when the ball is not visible, ensuring that the broadcast remains focused on the most relevant area of the pitch.

1. SETUP

```
load YOLO model
FOR each of 3 video files:
      start background thread to read frames
      store frames in a queue (max 10)
set score_history[3][5] = 0
set current_best = 0
```

2. FUNCTION get_detections(frame)

```
shrink frame to 1280px wide
run YOLO on shrunken frame
FOR each detected object:
      rescale coordinates to original size
      IF player AND height < threshold:
            save center (cx, cy)
      IF ball:
            save bounding box
return player_centers, ball_boxes, detections
```

2) Results and Analysis:

While the initial results were promising when the ball was clearly visible, this condition was rarely met during continuous play. The secondary criterion individual player proximity to the camera proved unreliable; isolated players, such as defenders moving toward the touchline far from the actual play, would inadvertently trigger a camera switch, leading to a loss of focus on the main action.

To rectify this, we replaced the proximity metric with a player density concentration analysis. This approach proved far more robust, particularly during static phases such as rucks, as it ensures the system remains locked on the dense cluster of players where the action is concentrated, rather than following peripheral movements.

Following the validation of this logic on static images, the system was implemented for video processing at a rate of two analyses per second (2 FPS). This frequency was chosen as a strategic compromise to minimize computational load while maintaining acceptable switching fluidity. However, despite these optimizations, the algorithm's overhead remains significant for our limited hardware resources. Consequently, while the logic is functionally validated, the real-time performance results are currently only partially verified under our existing hardware constraints.

3) Discussion:

3. FUNCTION compute_score(players)

```
IF fewer than 3 players:
      return 0   // camera ignored
spread = avg(std(x), std(y))  of player positions
    // low spread = grouped players = action
density = max(0,  500 - spread)
bonus   = count(players)  x  10
return  density + bonus
```

4. MAIN LOOP

```
LOOP:
    FOR each camera:
        get next frame from queue
        IF empty: frame = NULL
    IF all frames NULL: stop  // videos ended
    FOR each camera i:
        IF frame NULL:
            score = last known score; skip
        IF frame should be skipped:
            // every other frame skipped
            score = last known score; skip
        run get_detections(frame)
        draw detection boxes on frame
        IF ball found:
            pick ball closest to frame center
            score = 1000 - dist(ball, center)
            // ball priority, max score 1000
        ELSE IF >= 3 players found:
            score = compute_score(players)
            // density-based, max ~500+
        ELSE:
            score = 0
        add score to history (keep last 5)
        smooth_score = mean(history[i])
```

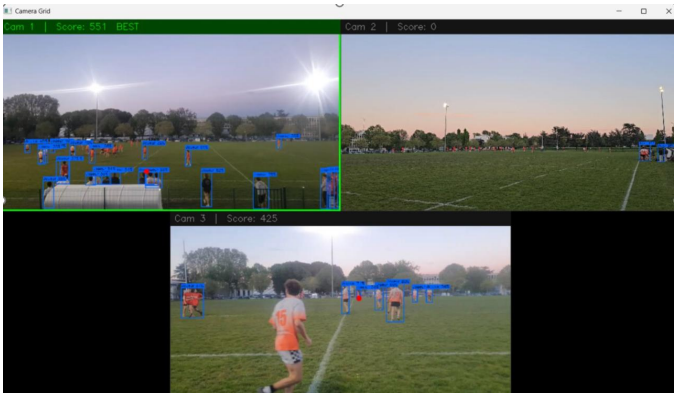
5. SWITCH DECISION

```
new_best = camera with highest smooth_score
IF new_best != current_best:
    IF score[new_best] > score[current] + 100:
        current_best = new_best
        // gap of 100 prevents oscillation
```

6. DISPLAY

```
show frame from current_best
label with camera number and score
IF Escape pressed: stop
```

```
END LOOP  ->  close all windows
```



The proposed algorithm has demonstrated high precision in action detection and camera switching logic. Recent testing phases have yielded highly conclusive results, confirming that our scoring heuristic effectively identifies the core of the game. However, a significant performance bottleneck occurs during video processing due to hardware limitations. On standard desktop environments, the system struggles to maintain the necessary inference speed for real-time responsiveness. While reducing the sampling rate (frames per second) could alleviate the computational strain, it would severely degrade the switching fluidity, often leaving the broadcast focused on an outdated area of play. Consequently, the transition from a functional prototype to a production-ready system necessitates high-performance computing resources such as dedicated GPUs or specialized edge-AI hardware capable of handling multiple high-definition video streams with minimal latency.

Conclusion

This work demonstrates that an automated multi-camera switching system for rugby match coverage can be designed and validated with minimal hardware resources. By combining a lightweight object detection model (YOLOv8n) with a density-based camera scoring algorithm, we were able to implement and test the core logic of the pipeline on consumer-grade equipment.

The principal limitations encountered throughout this project were hardware-driven rather than conceptual. Real-time video transmission was investigated but could not be tested due to the absence of dedicated equipment. For the same reason, validation on live video streams was not achievable, and the decision was made to evaluate the switching logic on static images instead, in order to at least verify that the underlying approach was sound. On the detection side, a rugby-specific trained model was identified but remained inaccessible as it was privately held, which led us to fall back on the generic YOLOv8n model trained on the COCO dataset.

Despite these constraints, the results obtained on static images confirm that the core logic of the system functions as intended. Player and ball detection operated correctly on real match footage, and the density-based scoring metric produced coherent camera rankings across the tested scenarios.

These outcomes suggest that the gap between the current prototype and a production-ready implementation is primarily a matter of resources rather than algorithmic design. With access to adequate hardware, real-time transmission and live video validation become achievable. Similarly, access to a domain-specific detection model would significantly improve detection accuracy on rugby-specific objects. With moderate investment in both equipment and model development, the system presented here provides a credible and low-cost foundation for automated sports broadcast in amateur rugby.